TechPost ELEVATED

DC – AI – Routing – Switching – Security – Wireless – Automation

Context File Optimization and Alerting Strategies

An HPE Juniper Networking Techpost

Christian Graf

2026-03-23

Contents

1 Context File Optimization and Alerting Strategies 2
1.1 Introduction . . . 2
1.2 Essential First Step: Contract Linting . . . 2
1.3 Why Lint First? . . . 3
1.3.1 Contract Lint . . . 3
1.3.2 Iterative Improvement with LLM Assistance . . . 3
1.4 Adding a Missing Requirement . . . 4
1.4.1 Best Practices for LLM-Assisted Improvement . . . 4
1.5 Advanced Topic: The HOW/WHAT/DELTA Framework . . . 5
1.6 The Concept . . . 5
1.7 Why This Approach Is Not Recommended . . . 5
1.8 Recommendation . . . 6
1.9 Defining Alerts in the Context File . . . 6
1.10 Reporting Results: Tables vs Lists . . . 7
1.11 Tables for Structured Summaries . . . 7
1.12 Lists for Failure Detail . . . 7
1.13 Summary . . . 8
1.14 Glossary . . . 9

1 Context File Optimization and Alerting Strategies

Christian Graf - 03/23/2026

Third part of the Series on the Context-Files and JMCP, covering tokens, data formats, and memory management.

Series Navigation:

- Part 1: Introduction & Basics
- Part 2: Tokens & Memory
- Part 3: Optimization & Alerting <- You are here
- Part 4: File Management
- Part 5: Redis & Performance
- Part 6: Hybrid Architecture

1.1 Introduction

In Parts 1 and 2, we covered context file fundamentals and token management. In this part, we'll explore advanced optimization techniques that can dramatically reduce context file size while maintaining functionality, and we'll examine the best practices for alerting.

LLM native built-in knowledge

Modern LLMs like Claude Sonnet, ChatGPT and Gemini bring extensive built-in networking knowledge to the table. When you ask them to check ISIS database synchronization, they already know the workflow without you spelling it out. A simple prompt like **Check if ISIS database is synced** triggers comprehensive workflows without detailed instructions, for example:

- Identify target routers and how to reach them
- Collect ISIS adjacency + LSDB state
- Normalize outputs (LSP IDs, sequence numbers, checksums)
- Compare LSDB views across routers
- Flag deltas (missing LSPs, mismatched seq/checksum)
- Pull supporting context if not synced
- Summarize sync status with specific differences

The LLM can natively perform the above steps. Capabilities like parsing CLI output (text, JSON, XML), extracting and normalizing fields, building data structures for comparison, comparing values across routers, and generating reports in table or summary format are all built into the LLM. Executing commands on actual routers requires external MCP tooling like either the vanilla Juniper MCP and the authors enhanced JMCP.

1.2 Essential First Step: Contract Linting

Two categories of prompts are useful for improving context files: one that strengthens quality, and one that reduces size. They serve different purposes and are not interchangeable. The first prompt we focus on is quality - the Contract Lint prompt.

1.3 Why Lint First?

When writing context files, it's easy to make assumptions about what's "obvious", but what's clear to you may be ambiguous to the LLM. The Contract Lint prompt systematically checks for:

- Missing numeric thresholds (e.g., "high CPU load" without defining the percentage)
- Ambiguous severity mappings (e.g., "flag if out of sync" without specifying WARNING vs CRITICAL)
- Incomplete recheck logic (e.g., "retry later" without specifying timing or escalation)
- Undefined alert formats (e.g., "report failures" without message template)

The Contract Lint prompt strengthens quality with no goal of reducing size. This is a mandatory step before any compression. This is the most uniquely valuable prompt in the toolbox. No simple paraphrase replaces it. It catches missing thresholds, undefined wait times, ambiguous scope rules, and logical gaps that would cause inconsistent LLM behavior at runtime. Apply it first, always.

1.3.1 Contract Lint

```
Review this context snippet like a linter.

Task:

List any ambiguity that could lead to inconsistent results (e.g., undefined wait time, instance/
    level selection, pseudo-node handling).
For each issue, propose a single crisp line to add or edit.
Do not rewrite the whole snippet.

Snippet:
<paste>
```

Always run Contract Lint before attempting compression or optimization. Compression prompts can mask ambiguity by removing verbose text that exposed the problem, lint catches issues while they're still visible.

1.3.2 Iterative Improvement with LLM Assistance

Once you have a clear, validated context file (after linting), the LLM itself can help you refine and extend it. This allows you to describe requirements in natural language and have the LLM translate them into crisp, structured entries - concise bullet points with clear thresholds, no verbose prose. The first prompts of removing bloat and help sharpening the content are by far the most impacting ones. Care is required when asking the LLM to remove or condense content. Add some constraints to it, like preserve thresholds,… It is a good idea to apply such prompts only to smaller sections, to keep area of changes low. This simplifies the review.

Common Improvement Prompts

For most context files, simple natural-language prompts work well and are safer than rigid structured rules:

```
# Remove bloat (simple)
"Remove pseudocode blocks and procedural descriptions. Preserve all thresholds, severities,
    alert formats, constraints, and wait times exactly as written. Do not change any numeric
    values or conditions."

# Remove bloat (alternative)
"Condense this section without changing intent or quality."

```

```
# Sharpen content
"Make this section more concise and clearer. Preserve all workflow steps, thresholds, numeric values, alert message templates with variables, severity levels, reporting formats, and constraint logic exactly as written. Only improve wording and remove redundant phrasing."

# Add missing logic
"Add fallback logic when cache is expired: query via JMCP, update cache, proceed with analysis. Do not modify existing thresholds or alert conditions."

# Fix ambiguity
"The recheck threshold doesn't specify what to do if the second check also fails. Add clear escalation guidance without changing existing threshold values or severity levels."

# Improve reporting
"Change reporting format to: table summary for multi-router status, then list format for each failure detail. Keep all existing alert message templates and field names unchanged."
```

For heavily bloated context files with extensive pseudocode blocks or multi-paragraph workflow descriptions, see the HOW/WHAT/DELTA Framework section below.

1.4 Adding a Missing Requirement

The natural language example below is verbose—as are most ad-hoc requirements. While context files allow considerable flexibility in structure, it's best to keep them concise. Always feel free to express requirements in natural language, but instead of adding verbose descriptions directly to your context file, ask the LLM to refine and structure them into concise, unambiguous specifications. The LLM understands network domains, routing protocols, and Junos. If uncertain whether a requirement is adequately covered, use Contract Lint to verify. Phrase your request naturally, then ask the LLM: "Add this to the context file. Keep it crisp and sharp."

```
Please factor this requirement as well:
There shall be one more check added: assume there is a LSP Sequence Number mismatch.
But we know that such mismatch can happen due to normal LSP propagation time.
It then makes sense to await the already detected Lifetime variance and then re-do the test.
Unless the LSP in question did not get a further update in the meantime, the LSP sequence-number shall be in sync.
Please keep it crisp and sharp.
```

LLMs have different quality and knowledge, but e.g. Claude Sonnet 4.6 would translate above natural language into a more crisp prompt as shown below. For such small examples the gain does not seem massive. But once a context-file reaches >100 lines, the operator realizes that navigating through the file becomes more demanding. Any reduction in size, while not sacrificing quality, helps for navigating, extending and maintaining.

```
- Seq delta = 1 (transient candidate): wait for the detected lifetime-variance window of that LSP (<=60s), then recheck:
  - No new LSP update observed + still mismatched -> CRITICAL
  - New LSP update arrived during wait -> recheck once more; if still mismatched -> CRITICAL; if resolved -> PASS
```

With the LLM's assistance, the verbose natural language above becomes a complete two-branch recheck rule, no propagation excuse remains unhandled.

1.4.1 Best Practices for LLM-Assisted Improvement

A few context-file-specific cautions:

- **Always lint before compressing**: run the Contract Lint prompt (Lint) first; compression prompts can mask ambiguity by removing the verbose text that exposed it

- **Review every compression result**: a fused WHAT+DELTA line can look like removable scaffolding; verify nothing was dropped
- **Resist scope creep**: LLMs tend to add checks when asked to improve; reject additions that aren't in your operational contract

1.5 Advanced Topic: The HOW/WHAT/DELTA Framework

Note: This section describes an experimental optimization approach that is not recommended for production use. It's included for completeness, but the simple compression prompts shown earlier are safer and more practical.

1.6 The Concept

LLMs have built-in knowledge of networking protocols and generic engineering tasks. Theoretically, you could compress context files by removing content the LLM "already knows":

- **HOW**: Generic procedural steps (execute commands, parse output, build data structures), the LLM knows them
- **WHAT**: Protocol knowledge from training (IS-IS field names, standard checks, flooding behavior), the LLM learned this
- **DELTA**: Environment-specific rules the LLM cannot infer (your thresholds, severities, alert templates, constraints), you must specify them

For example, in an IS-IS LSDB check: lines like "Parse LSP entries: extract LSP-ID, sequence number, checksum" are pure HOW, the LLM knows how to parse. But "Seq delta >=2 -> CRITICAL" is DELTA, only you know your threshold.

1.7 Why This Approach Is Not Recommended

While theoretically appealing, this optimization strategy has significant practical problems:

- 1. **LLM Dependency**: Optimized files rely heavily on current LLM capabilities. If you switch from Claude to ChatGPT, or if a model update changes built-in knowledge, your context files may break silently. What "the LLM knows" varies across models and versions.
- 2. **Modest Savings**, High Risk: In practice, most content is WHAT+DELTA fused together. The line "Seq delta >=2 -> CRITICAL" names both the check (WHAT) and your threshold (DELTA), you cannot separate them. Only pure HOW lines can be removed, typically 15-25% of verbose first drafts. The simple "remove procedural descriptions" prompt achieves similar savings with less risk.
- 3. **Maintenance Burden**: Heavily optimized files become cryptic. New team members or future-you reviewing them may incorrectly assume steps are missing. The compression makes files harder to audit and maintain.
- 4. **Brittle Optimization**: Over-optimizing creates a maintenance trap. When requirements change, you must carefully add back context. It's often faster to maintain slightly more verbose files that remain self-documenting.

1.8 Recommendation

For production context files, use the simple compression prompts from earlier in this post:

- "Remove pseudocode blocks and procedural descriptions. Preserve all thresholds, severities, alert formats, constraints, and wait times exactly as written."
- "Make this section more concise and clearer. Preserve all workflow steps, thresholds, and constraint logic exactly as written. Only improve wording."

These achieve 80% of potential savings with 20% of the risk. Save HOW/WHAT/DELTA analysis for theoretical discussion, not production optimization.

1.9 Defining Alerts in the Context File

When writing alert specifications in your context file, the rules that tell the LLM when to fire an alert and what to say, format matters for both maintainability and token efficiency.

Three format options exist for defining alert rules:

- **Natural language**: Descriptive prose explaining conditions and actions
- **Tables**: Structured rows with condition, severity, and message columns
- **Lists**: Bullet-point format with structured entries

Natural language is straightforward to write but verbose and ambiguous. Reading lengthy rule descriptions requires careful attention to catch every condition.

Tables improve parsability but become unwieldy when alert messages vary in length or contain multiple {variables}, columns become uneven and hard to scan.

Lists are recommended for alert definitions. They offer the best balance:

- The LLM benefits from fewer tokens required
- Humans benefit because lists are easier to maintain, extend, and read

Each alert rule follows this structure:

```
- <condition>: <severity> --- "<message template>"
```

Why tables don't work well for alert definitions: Message templates vary in length and contain multiple **{variables}**, making columns uneven and hard to scan. The value-add of alignment disappears as cells grow. A list line scales naturally no matter how long the template gets; a table cell does not.

Here's how a table of alert rules would look:

Less preferred (table format):

```
| Condition | Severity | Message |
|-------------------|----------|-------------------------------------------------------------------------|
| Seq delta >=2 | CRITICAL | LSDB out of sync detected on {instance}/{level} - LSP {lsp_id}:
    router {router1} reports sequence 0x{seq1} (checksum {chk1}), but router {router2} reports
    sequence 0x{seq2} (checksum {chk2}). Delta exceeds threshold. This may indicate flooding
    issues or database corruption requiring investigation. |
| Missing LSP | CRITICAL | LSP {id} missing on {router} |
| Lifetime var >60s | WARNING | Propagation delay detected - LSP {id} lifetime variance {delta}s
     exceeds threshold on routers {router_list}. Check adjacency state and retransmit counters.
    |
```

The same rules as a list are easier to extend and maintain. Lists make the message templates clearer for humans while remaining efficient for the LLM.

Preferred (list format):

```
- Seq delta >=2: CRITICAL --- "LSDB out of sync detected on {instance}/{level} - LSP {lsp_id}:
    router {router1} reports sequence 0x{seq1} (checksum {chk1}), but router {router2} reports
    sequence 0x{seq2} (checksum {chk2}). Delta exceeds threshold. This may indicate flooding
    issues or database corruption requiring investigation."
- Missing LSP: CRITICAL --- "LSP {id} missing on {router}"
- Lifetime variance >60s: WARNING --- "Propagation delay detected - LSP {id} lifetime variance {
    delta}s exceeds threshold on routers {router_list}. Check adjacency state and retransmit
    counters."
```

1.10 Reporting Results: Tables vs Lists

Once you've defined when alerts should fire, the next question is: how should the LLM present the findings? The format choice has a meaningful impact on token usage and readability.

1.11 Tables for Structured Summaries

Tables are the right choice when reporting structured data across multiple entities, routers, interfaces, LSPs. They use 30–50% fewer tokens than equivalent bullet lists for the same data, and make comparative analysis easy to read at a glance. Prompt: **Report the ISIS queue depth as a list** List format (higher token cost):

```
**R1 - ge-0/0/1.0:**
- High priority xmit queue: 0
- Normal priority xmit queue: 0
- Low priority xmit queue: 2

**R1 - ge-0/0/2.0:**
- High priority xmit queue: 0
- Normal priority xmit queue: 1
- Low priority xmit queue: 0

**R2 - ge-0/0/1.0:**
- High priority xmit queue: 0
- Normal priority xmit queue: 0
- Low priority xmit queue: 0
```

Tables are often easier to read and have a lower token cost for the same data.

Prompt: **Report the ISIS queue depth as a table**

```
- Low priority xmit queue: 0
```

Use tables for reporting when: data spans 5+ entities, fields repeat consistently, or the goal is comparative analysis or a status dashboard.

1.12 Lists for Failure Detail

Lists are better for failure drill-downs on a single entity, where the data is nested, narrative, or variable in structure.

The example below is **not something you define in a context file**: it is what the LLM produces autonomously. When a threshold is breached, the LLM cross-correlates supporting diagnostics (adjacency state, retransmit counters) and infers a root cause on its own. That kind of structured reasoning is built-in LLM capability; you

don't script it. What you do specify in the context file is the format instruction: "**FAIL detail: for each failed router, list format with supporting diagnostics**".

```
**R1 - ge-0/0/1.0 - CRITICAL:**
- Seq mismatch detected: 0x15f -> 0x160 (delta: 1)
- Rechecked after 30s: still mismatched (persistent)
- Supporting data:
  - Adjacency: UP (neighbor R2)
  - Retransmits: 15 (1.5% of sent)
  - Drops: 0
- Root cause: Likely slow propagation on R2 side
```

Use lists when: reporting on a single failing entity, presenting nested supporting diagnostics, or providing a narrative root-cause. One line in your context file is enough to unlock this behavior, **FAIL detail: for each failed router, list format with supporting diagnostics**, the cross-correlation and root-cause inference happen automatically.

Hybrid: Table Summary + List Drill-Down

The most effective pattern combines both: a table for the high-level status overview, followed by list-format detail for each failure.

Instruct the LLM in your context file like this:

```
**Reporting:**
- PASS: one-line summary --- "LSDB synced across all routers in {instance}/{level}"
- FAIL summary: table --- Router | LSDB Sync | Adjacency | Queue Depth | Status
- FAIL detail: for each failed router, list format with supporting diagnostics
```

Which produces output like:

```
## ISIS Health Summary

| Router | LSDB Sync | Adjacency | Queue Depth | Status |
|--------|-----------|-----------|-------------|----------|
| R1 | [x] | [x] | [x] | PASS |
| R2 | [x] | [x] | [x] | PASS |
| R3 | [ ] | [x] | [x] | CRITICAL |

**R3 - LSDB Sync - CRITICAL:**
- Missing LSP: R2.00-00 (seq: 0x15f) not present on R3
- Persistent after 30s recheck
- Supporting checks:
  - Adjacency R3-R2: UP
  - No retransmits or drops
- Recommendation: Check ISIS database manually on R3
```

1.13 Summary

In this part, we covered:

- **Contract Lint as the essential first step**: catch ambiguity and missing constants before they propagate
- Using LLMs to iteratively improve and extend your context files with natural language
- The 2-prompt approach: Contract Lint (quality) + compression prompts (size)
- Advanced optimization theory: distinguishing HOW/WHAT/DELTA to identify redundant content
- Defining alerts in the context file: always lists, never tables
- Reporting results: tables for summaries, lists for failure detail, hybrid for both

In Part 4, we'll explore file management strategies: Phase 0 data collection, master index patterns, and the Extract/Store/Reuse workflow for large-scale network automation.

Coming up in Part 4:

- Phase 0: Data collection and caching
- Master Index pattern for temporal consistency
- Extract/Store/Reuse workflow
- File saving strategies and naming conventions
- Split workflows for token optimization

1.14 Glossary

- Barrier-sync: Executing commands simultaneously across all devices by establishing all connections first, then firing commands at the same time, ensuring temporal consistency in collected data.
- Contract: The environment-specific specifications in a context file (thresholds, constraints, alert formats) that define how the LLM should behave for your specific network.
- Contract Lint: A prompt pattern that reviews a context-file snippet for ambiguity and missing constants, undefined wait times, unscoped instance/level rules, missing escalation paths, and proposes crisp fixes. Applied before compression, not instead of it.
- DELTA: The minimum required content in a context file: environment-specific rules the LLM cannot infer on its own, your thresholds, severity mappings, alert message templates, recheck rules, scope constraints, and tooling requirements (e.g., barrier-sync). Everything else is potentially removable.
- Delta-contract: A context-file snippet that contains only DELTA elements, no procedural scaffolding, no protocol knowledge the LLM already has. The goal artifact of the HOW/WHAT/DELTA optimization process.
- Delta-only approach: Context file optimization strategy that removes generic procedures (HOW) and protocol knowledge (WHAT) the LLM already knows, keeping only environment-specific requirements.
- HOW: Generic procedural steps (execute, parse, compare, generate) that skilled LLMs already understand and can infer without explicit instructions.
- IS-IS (Intermediate System to Intermediate System): A link-state interior routing protocol used in large IP networks. Routers exchange LSPs to build a synchronized topology database (LSDB). Used as the primary example protocol throughout this post.
- IS-IS LSDB (Link State Database): The topology database maintained by each IS-IS router, built from received LSPs. All routers in the same area should hold an identical LSDB; divergence indicates a synchronization problem.
- List format: Alerting data presented as bullet points. More human-readable but uses 30-50% more tokens than tables for structured data.
- LSP (Link State PDU): In IS-IS routing protocol, Protocol Data Units containing routing and topology information. Key fields include LSP-ID, sequence number, checksum, and lifetime.
- MCP (Model Context Protocol): A protocol that allows LLMs to invoke external tools, such as executing commands on network devices, through a defined interface. LLMs orchestrate MCP tool calls but cannot execute commands natively without an MCP server present.
- Procedural scaffolding: The surrounding prose in a context file that describes how to execute a check, the command to run, how to parse the output, how to build a comparison structure. This is pure HOW content and the primary token-reduction target; it can be removed without affecting the contract.
- Signal-to-token ratio: Measure of information density in context files. Higher ratio means more useful specifications per token consumed.
- Table format: Alerting data presented in markdown tables. More compact (30-50% fewer tokens) and better for comparative analysis across multiple entities.

- Threshold: Specific numeric or conditional values that define when alerts should trigger (e.g., “seq delta >=2 = CRITICAL”).
- WHAT: Domain-specific protocol knowledge (standard fields, checks, diagnostics) that LLMs learn during training and don’t need explicitly defined in context files.
- WHAT+DELTA (fused line): A context-file bullet where protocol knowledge (WHAT) and an environment-specific rule (DELTA) cannot be separated, for example, Seq delta >=2 -> CRITICAL. Naming the check and specifying the threshold are a single inseparable statement. These lines must be preserved; compression prompts must not treat them as removable WHAT.



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.
Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.
Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com



